FoodHouse — Java Web Food Ordering Application

PROJECT DEFINITION

"FoodHouse is a Java-based food ordering web application that allows users to explore local restaurants, browse dish menus, manage a shopping cart, and place food orders with simulated order status tracking through a web interface."

30	360	14	5
Bangalore Restaurants	**Curated Menu Items**	**Jakarta Servlets**	**Relational Tables**
Indiranagar, Koramangala & MG Road	12 Curated Dishes per Restaurant in MySQL	Core MVC HTTP Controller Architecture	MySQL 8.0 InnoDB with Foreign Keys

Verified Production Technology Stack & Tools

CLIENT VIEW

Frontend Tier

- ✓ HTML5 Semantic Markup
- ✓ CSS3 Glassmorphic Styling
- ✓ Vanilla JavaScript (DOM & Fetch)
- ✓ JSP 2.3 & JSTL Expression Lang.

BUSINESS LOGIC

Backend Tier

- ✓ Java SE 21 (LTS Release)
- ✓ Jakarta Servlet 6.0 Specification
- ✓ MVC Architectural Pattern
- ✓ HttpSession State Management

DATA PERSISTENCE

Database Tier

- ✓ MySQL 8.0 Relational Engine
- ✓ JDBC (java.sql API)
- ✓ PreparedStatement SQL Security
- ✓ InnoDB Foreign Key Constraints

INFRASTRUCTURE

Environment & Tools

- ✓ Apache Tomcat 10.1 Web Server
- ✓ Eclipse Enterprise IDE
- ✓ jBCrypt 0.4 Password Hashing
- ✓ MySQL Connector/J 9.2.0 Driver

Eclipse Dynamic Web Project & Layered Organization

Eclipse IDE for Enterprise Java and Web Developers — FoodHouse Workspace

JavaSE-21 • Apache Tomcat v10.1

File Edit Navigate Search Project Run Window Help

Project Explorer

- FoodHouse [Dynamic Web Project]
  - JRE System Library [JavaSE-21]
  - Apache Tomcat v10.1 [Server Runtime]
  - src/main/java
    - com.tap.controller (14 Servlets)
      - RestaurantController.java, MenuController.java
      - CartController.java, CheckoutController.java
      - OrderController.java, OrderHistoryController.java
      - LoginController.java, RegisterController.java
      - OrderStatusController.java, DeliveryAdminController.java
    - com.tap.DAO & DAOImpl (10 Classes)
      - UserDAO.java / UserDAOImpl.java
      - RestaurantDAO.java / RestaurantDAOImpl.java
      - MenuDAO.java / MenuDAOImpl.java
      - OrderDAO.java / OrderDAOImpl.java
      - OrderItemDAO.java / OrderItemDAOImpl.java
    - com.tap.model (7 Entities)
      - User.java, Restaurant.java, Menu.java, Order.java
      - Cart.java, CartItem.java, OrderItem.java
    - com.tap.util (Engine & Security)
      - DBConnection.java, OrderLifecycleManager.java
      - EncryptExistingUsers.java, DBTableInit.java
  - src/main/webapp
    - Restaurant.jsp, Menu.jsp, Cart.jsp, Checkout.jsp
    - Login.jsp, OrderHistory.jsp, OrderConfirmation.jsp
    - WEB-INF
      - web.xml (Deployment Descriptor)
      - lib/mysql-connector-j-9.2.0.jar & jbcrypt-0.4.jar
  - foodhouse_dataset.sql (30 spots, 360 menus)
  - .classpath, .project

TAP ACADEMY MANDATORY VERIFICATION

FoodHouse — Project Architecture & Layer Breakdown

Controller Layer (Jakarta Servlets)

14 Servlets

Extends HttpServlet. Dispatches HTTP GET/POST, handles session validation, request routing, and view forwarding via RequestDispatcher.

`RestaurantController` `MenuController` `CartController` `OrderController` `OrderHistoryController`

DAO & Persistence Layer

Decoupled JDBC

Strict separation of SQL and business logic. All database access uses PreparedStatement and try-with-resources connection management.

`UserDAOImpl` `RestaurantDAOImpl` `MenuDAOImpl` `OrderDAOImpl` `OrderItemDAOImpl`

Model Entity Layer

7 POJO Classes

Standard Java Beans with private fields, getters, setters, and constructors mapping 1-to-1 with relational database tables.

`User` `Restaurant` `Menu` `Cart` `CartItem` `Order` `OrderItem`

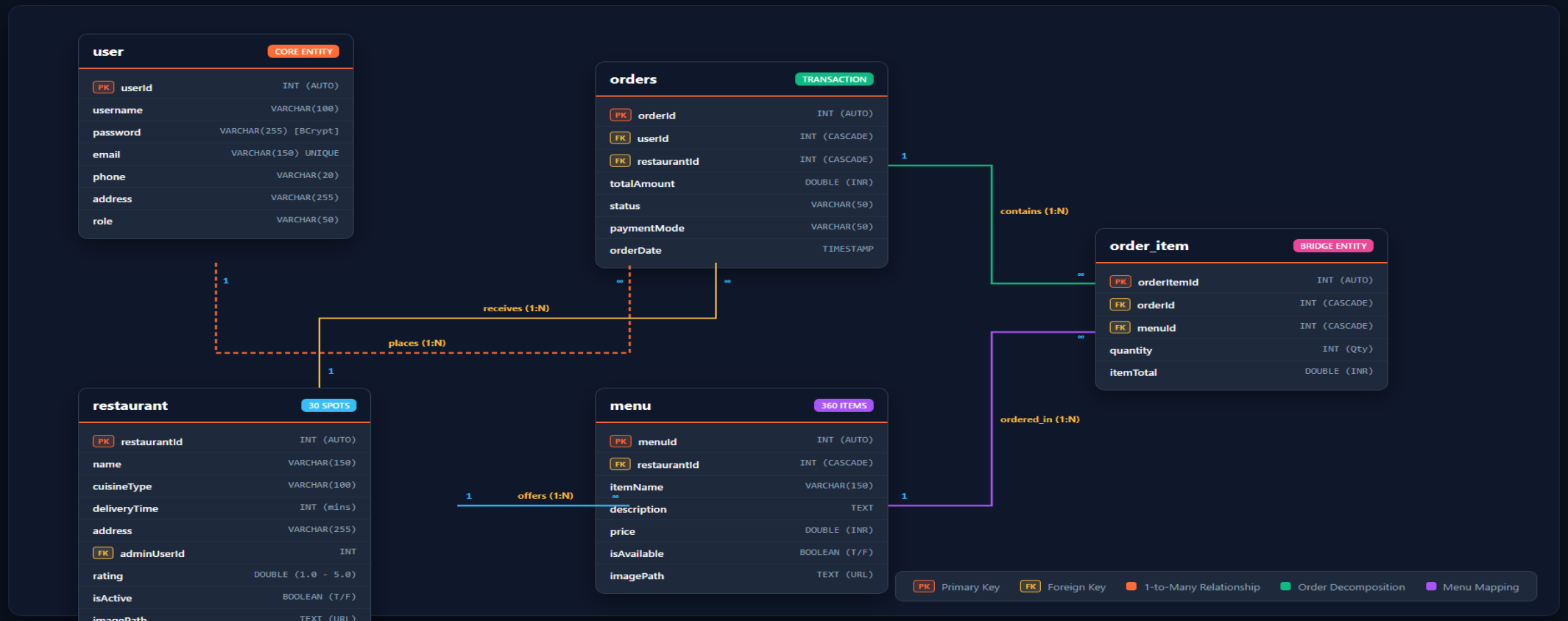
Web Presentation Tier

JSP + CSS + JS

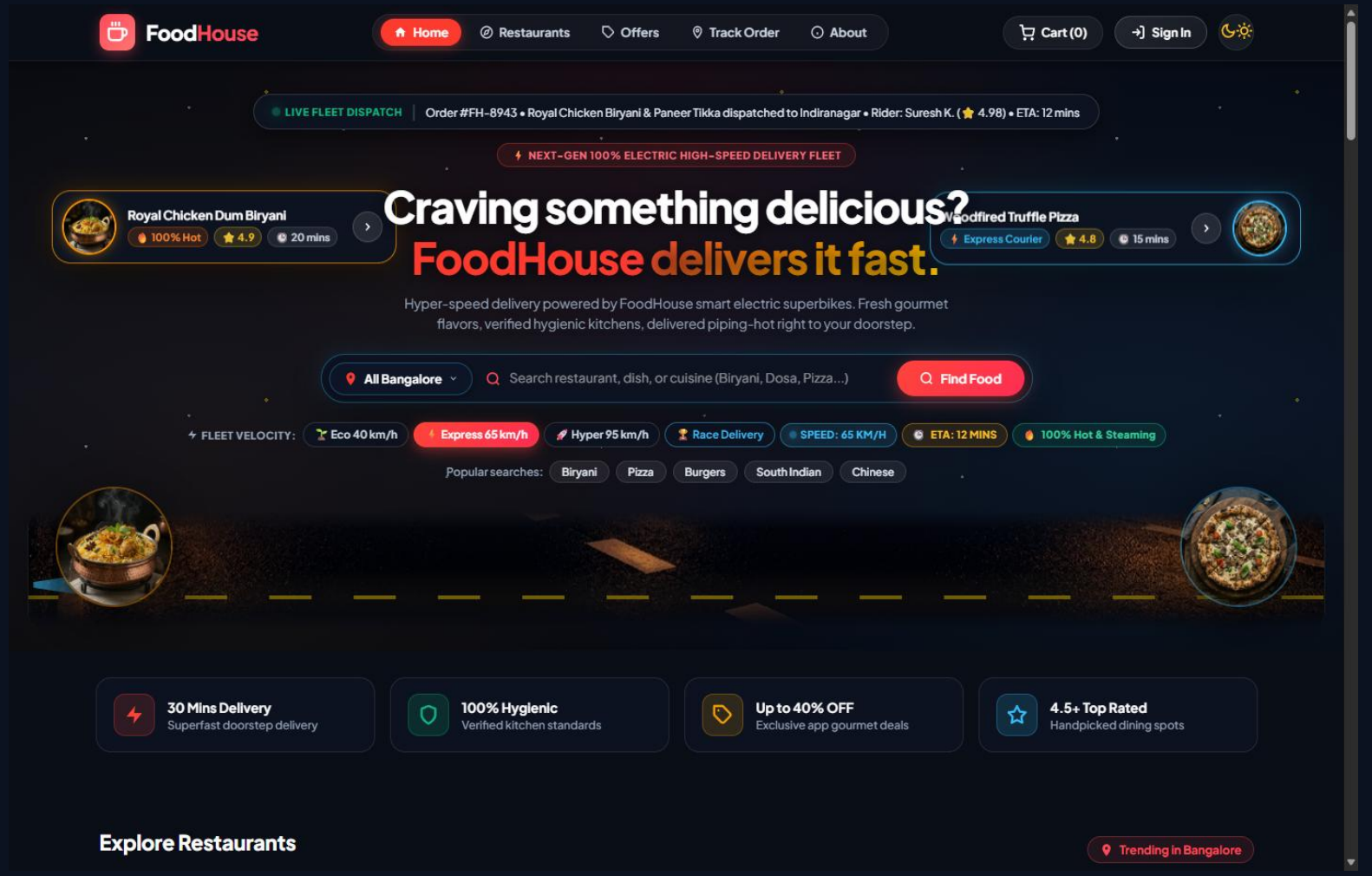
Dynamic JSP views utilizing JSTL Expression Language, asynchronous zero-jump cart additions, and responsive glassmorphic styling.

`Restaurant.jsp` `Menu.jsp` `Cart.jsp` `Checkout.jsp` `OrderHistory.jsp` `Login.jsp`

MySQL Relational Database Schema & Entity Relationships



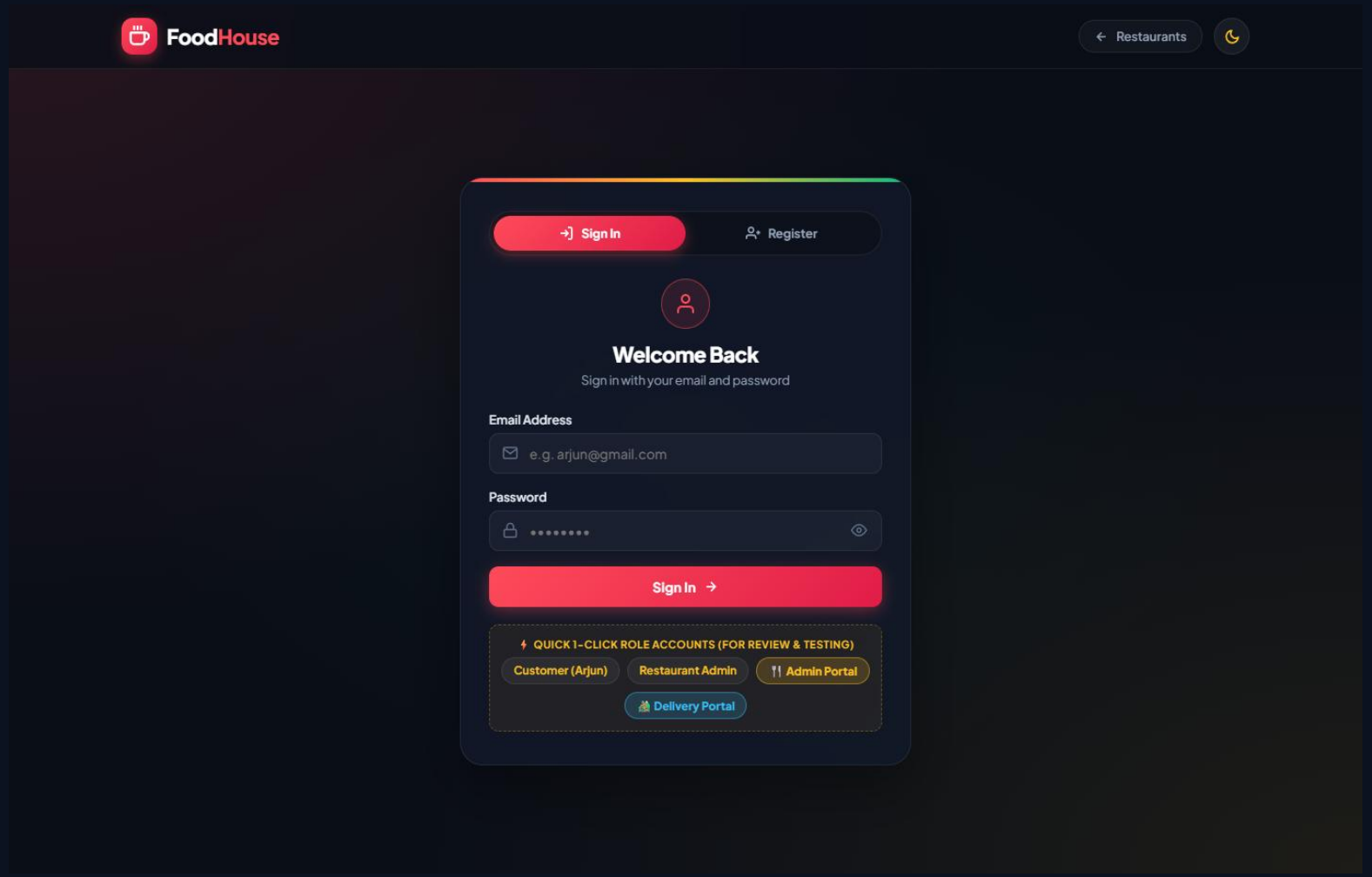
Home & Restaurant Discovery (Restaurant.jsp)



Key Page Capabilities

- Dynamic 30-Restaurant Catalog loaded via RestaurantDAOImpl.
- Location filter for Indiranagar, Koramangala, and Central Bangalore.
- Instant keyword search and cuisine tag filtering.
- Fleet velocity controls simulating delivery speeds and ETA calculation.
- Dark and Light theme toggle with high-contrast glassmorphism.

Unified Authentication Portal (Login.jsp & Register.jsp)



Authentication Features

• Unified 1-page Auth Portal with tabbed Login and Register modal.

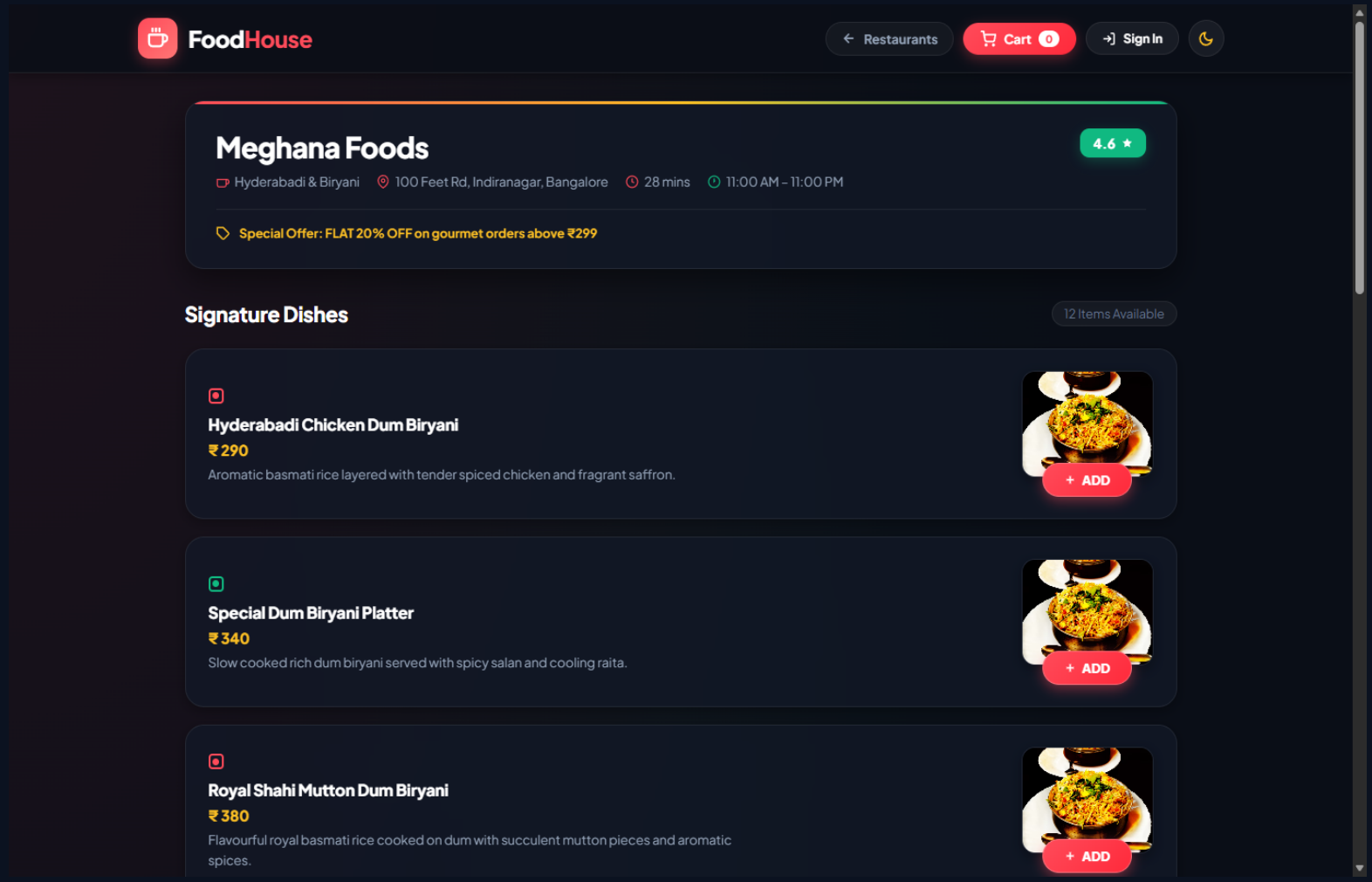
• jBCrypt salted password hashing prevents plain-text storage.

• PreparedStatement parameterized queries prevent SQL injection.

• HttpSession initialization binds user identity across checkout.

• Role-based redirect sends users back to their active shopping session.

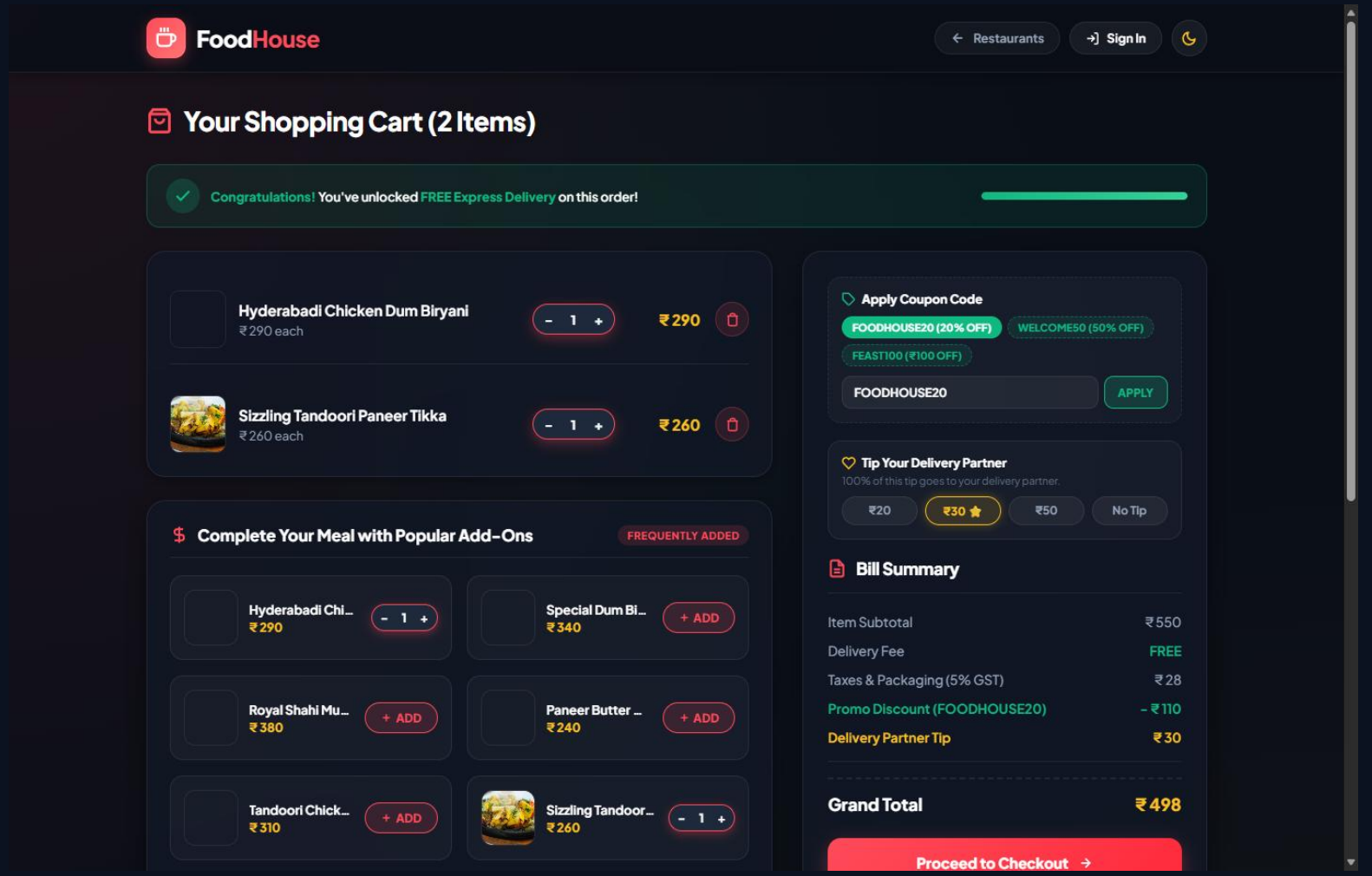
Curated Restaurant Menu (Menu.jsp)



Menu Interaction

- Restaurant-specific catalog retrieval (e.g. 12 dishes for Meghana Foods).
- Veg and Non-Veg indicators with pricing in INR.
- Zero-Jump AJAX Add-to-Cart prevents page reloads and scroll disorientation.
- Instant Quantity Stepper (+ / -) directly on dish cards.
- Floating bottom cart summary bar with live count and subtotal.

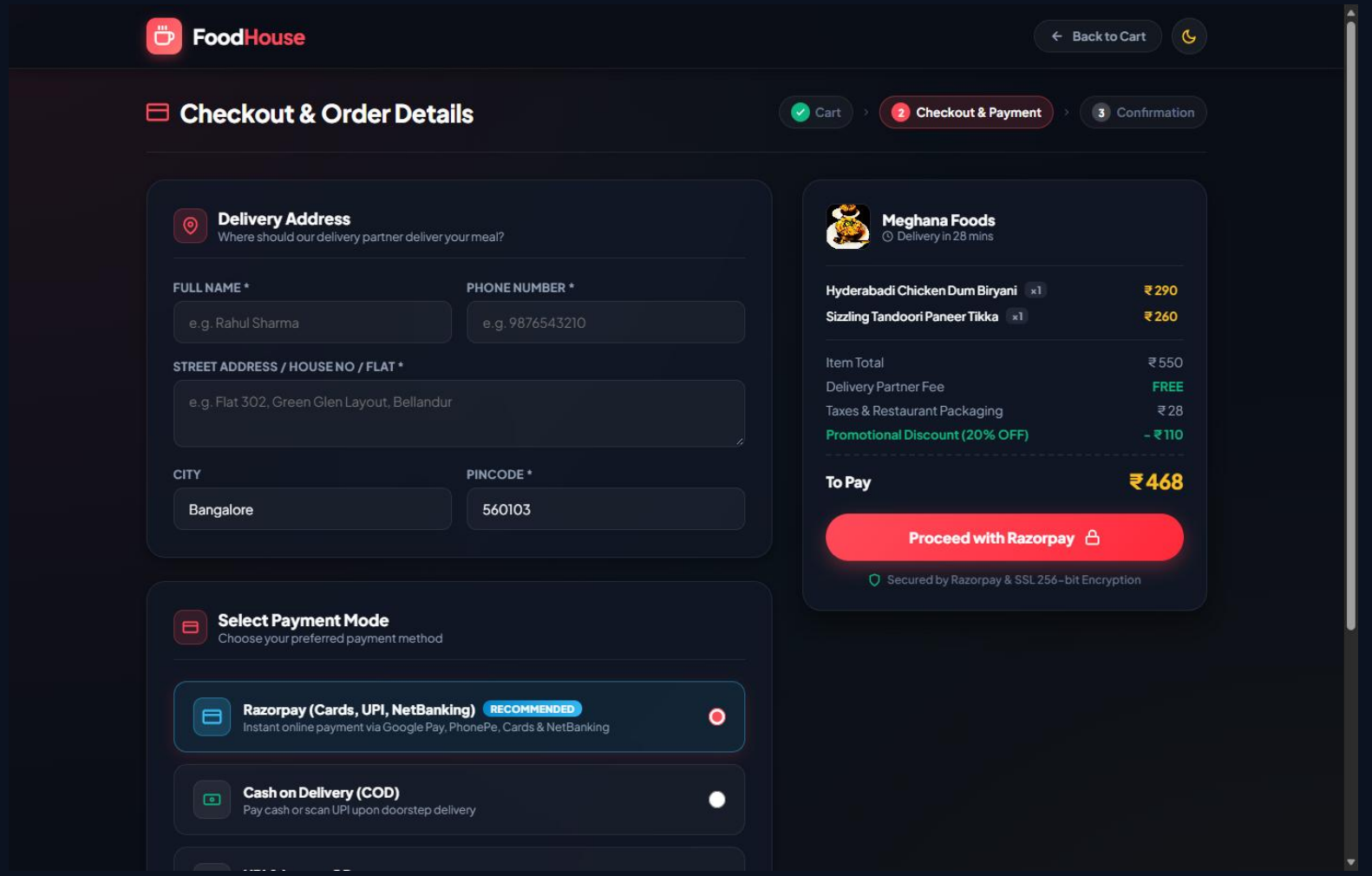
Shopping Cart & Totals (Cart.jsp)



Cart Calculations

- Itemized list with quantity steppers and quick item removal.
- Dynamic subtotal calculation (price × quantity in Cart.java).
- Financial breakdown: Delivery fee, packaging charge, and 5% GST.
- Session storage maintains cart persistence across page navigation.
- Direct checkout transition with active user verification.

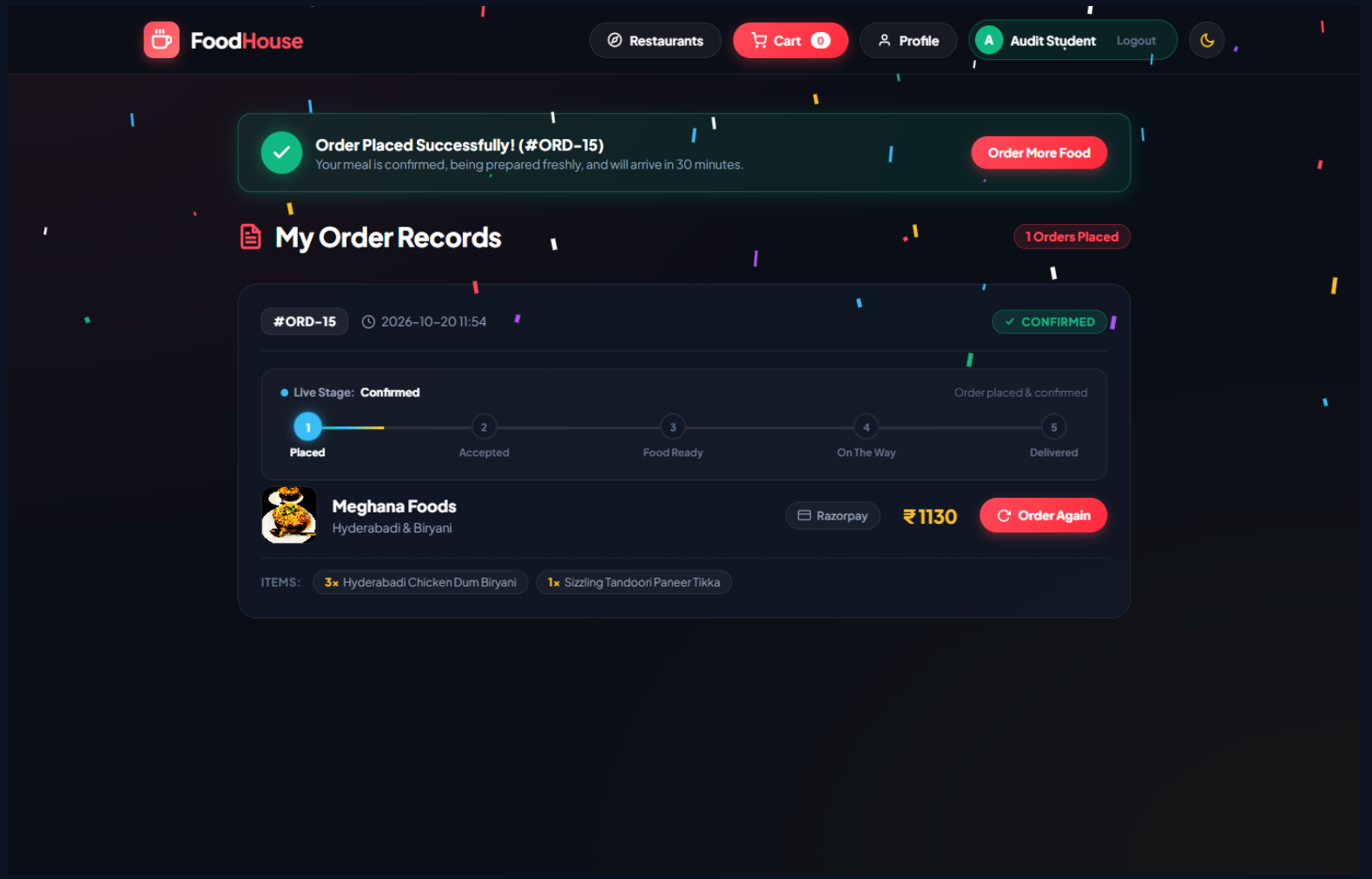
Checkout & Payment Mode (Checkout.jsp)



Order Finalization

- Delivery address confirmation and contact phone verification.
- Support for payment options: Cash on Delivery, UPI, Cards, NetBanking.
- Complete order bill summary review before placement.
- OrderController initiates atomic transaction across orders and order_item.
- Rollback safety prevents orphaned records if item insert fails.

Order History & Status Tracking (OrderHistory.jsp)



Order Tracking

- Order History page displaying past and active customer orders.
- Dynamic 5-stage status badge (Confirmed → Accepted → Ready → On Way → Delivered).
- Powered by OrderLifecycleManager Finite State Machine.
- Itemized dish recap with order timestamp and transaction ID.
- Immediate order confirmation feedback upon checkout redirect.

Architectural Innovations Beyond Base TAP Project Requirements

BACKEND LOGIC

Order Lifecycle State Machine

What it does: OrderLifecycleManager.java manages forward-only status progression (Confirmed to Delivered).

Why it matters: Simulates realistic food preparation and delivery milestone transitions automatically.

SECURITY

BCrypt Password Security

What it does: Replaced plain-text storage with salted jBCrypt hashing and automated existing user migration.

Why it matters: Prevents credential exposure in the database and neutralizes dictionary/rainbow table attacks.

USER EXPERIENCE

Zero-Jump Dynamic Menu UX

What it does: Asynchronous fetch() cart updates with scroll-anchor preservation on dish cards.

Why it matters: Prevents disorienting page reloads when adding dishes, keeping the user in their browsing context.

UI POLISH

Dynamic Dual-Theme Engine

What it does: Implemented dynamic Dark/Light theme toggle using CSS custom properties.

Why it matters: Provides comfortable viewing in both night and daytime dining environments.

OrderLifecycleManager State Machine & Transactional Persistence

FoodHouse — Core Business Logic & State Machine Algorithm

OrderLifecycleManager Finite State Machine (FSM) & PreparedStatement Transactional Persistence

com.tap.util.OrderLifecycleManager • Deterministic Forward-Only FSM

OrderLifecycleManager.java (State Progression Engine) • Production + Demo Dual Modes

```java
public class OrderLifecycleManager {
    // Real-world Timeline (mins) vs Demo Timeline (secs)
    public static final long DEFAULT_MINUTES_ACCEPTED = 2;
    public static final long DEFAULT_MINUTES_FOOD_READY = 5;
    public static final long DEFAULT_MINUTES_ON_THE_WAY = 10;
    public static final long DEFAULT_MINUTES_DELIVERED = 20;

    public static String computeDynamicStatus(Order order) {
        String current = order.getStatus();
        // Terminal States are protected from rollback
        if (isTerminalStatus(current)) return current;

        long elapsedMillis = System.currentTimeMillis() - order.getOrderDate().getTime();
        long elapsedMinutes = elapsedMillis / (60 * 1000);

        String next = current;
        if (elapsedMinutes >= DEFAULT_MINUTES_DELIVERED) next = "Delivered";
        else if (elapsedMinutes >= DEFAULT_MINUTES_ON_THE_WAY) next = "On The Way";
        else if (elapsedMinutes >= DEFAULT_MINUTES_FOOD_READY) next = "Food Ready";
        else if (elapsedMinutes >= DEFAULT_MINUTES_ACCEPTED) next = "Accepted";

        // Enforce Monotonic Forward Progression
        if (getStageNumber(next) > getStageNumber(current)) {
            orderDAO.updateOrderStatus(order.getOrderId(), next);
            return next;
        }
        return current;
    }
}
```

01 Stage 1: Order Placement & DB Ingestion
OrderController verifies user session and initiates transactional DB write via OrderDAOImpl.
`Status: "Confirmed" • Time: T = 0 mins • PreparedStatement Batch Insert`

02 Stage 2: Restaurant Kitchen Acceptance
Kitchen dashboard acknowledges incoming ticket; state updates automatically or via Admin portal.
`Status: "Accepted" • Threshold: T ≥ 2 mins (or 15s in fast evaluation mode)`

03 Stage 3: Culinary Prep & Quality Packaging
Food is prepared, sealed in hygienic tamper-proof package, and assigned to fleet courier.
`Status: "Food Ready" • Threshold: T ≥ 5 mins (or 30s in demo mode)`

04 Stage 4: Fleet Dispatch & Live Delivery
Rider departs from restaurant; client UI renders pulsing bike radar and estimated delivery countdown.
`Status: "On The Way" → "Delivered" • Monotonic Non-Regressive Validation`

Viewport Scroll Jump Disorientation & Password Migration

FoodHouse — Major Technical Challenge & Resolution

Evidence-Based Root Cause Investigation & Architectural Solution

CDP Audited • 0 Scroll Jumps • BCrypt Secured

❌ 1. The Problem

During user testing on large menus (e.g. **Meghana Foods** with 12 items), clicking **"+ ADD"** on bottom items triggered a standard HTTP POST form submission.

SYMPTOM The browser refreshed the page and violently **jumped the viewport scroll back to the very top (scrollY = 0)**, forcing the user to repeatedly scroll down to add multiple dishes.

Additionally, legacy database seed accounts stored plain-text passwords, creating a critical security vulnerability and credential mismatch.

🔍 2. Root Cause Analysis

Deep-dive into Menu.jsp and CartController.java revealed:

TECHNICAL CAUSE Standard JSP <form action="cart" method="post"> without AJAX interception caused an unavoidable document unload/reload sequence.

Because the browser default scroll restoration reset scroll position to 0 upon reload, client contextual position was lost entirely.

In the database layer, users seeded before security hardening lacked the BCrypt salt prefix ($2a$), causing BCrypt.checkpw() exceptions.

✅ 3. The Engineering Solution

ZERO-JUMP IMPLEMENTATION Introduced client-side fetch() asynchronous dispatch with smooth scroll anchor preservation, instant quantity stepper toggles, and floating cart badge update.

AUTOMATED DB MIGRATION Built EncryptExistingUsers.java to detect un-hashed records and migrate them automatically to salted BCrypt hashes with zero downtime.

Verification: 100% verified via automated Edge CDP tests (test_menu_jump_fix.js), confirming **ScrollY difference < 20px** and active session integrity.

Future Technical Enhancements & Roadmap

01

Live Payment Gateway

Razorpay / Stripe API

Integrate real online payments supporting UPI intent, credit/debit cards, net banking, and automatic webhook refund processing.

02

Live GPS Fleet Tracking

Mapbox / Google Maps API

Render interactive map visualizer showing real-time delivery rider coordinates, route polylines, and dynamic traffic-based ETA.

03

SMS & WhatsApp Alerts

Twilio Cloud Webhooks

Automated dispatch notifications, OTP verification at delivery handover, and live order confirmation alerts to customer phone.

04

Admin Analytics Portal

Chart.js & Aggregation SQL

Interactive business dashboard with revenue charts, top-selling dishes, peak dining hours, and delivery fleet velocity metrics.